

# Automating the Development and Management of Intelligent Software Systems

Vishal Palaniappan

DRAFT

# Contents

|           |                                                                |           |
|-----------|----------------------------------------------------------------|-----------|
| <b>1</b>  | <b>Introduction</b>                                            | <b>2</b>  |
| <b>2</b>  | <b>The Design</b>                                              | <b>2</b>  |
| 2.1       | Meaning of the World . . . . .                                 | 2         |
| 2.1.1     | Shared Meaning . . . . .                                       | 2         |
| 2.2       | Narrative of the World . . . . .                               | 4         |
| 2.3       | Semantic Invalidity . . . . .                                  | 4         |
| <b>3</b>  | <b>Semantic Reasoning Engine and Diagnosing Failures</b>       | <b>5</b>  |
| 3.1       | Meaning of Failures . . . . .                                  | 5         |
| 3.2       | Design Abstraction Language . . . . .                          | 5         |
| 3.3       | Types of Reasoning . . . . .                                   | 5         |
| 3.3.1     | Reasoning about Requirements . . . . .                         | 6         |
| 3.3.2     | Reasoning about Internal Consistency . . . . .                 | 6         |
| 3.3.3     | Reasoning about Reality . . . . .                              | 7         |
| 3.4       | Practical Considerations for Reasoning about Reality . . . . . | 7         |
| <b>4</b>  | <b>Computable Semantic Model</b>                               | <b>8</b>  |
| 4.1       | Implementing a Computable Semantic Model . . . . .             | 8         |
| 4.2       | Computable Semantic Primitives . . . . .                       | 8         |
| 4.3       | Automated Instrumentation . . . . .                            | 9         |
| <b>5</b>  | <b>Domain Specific Compression</b>                             | <b>10</b> |
| 5.1       | Compressed Log Processor . . . . .                             | 10        |
| <b>6</b>  | <b>Design Learning Platform</b>                                | <b>12</b> |
| 6.1       | Automated Development using the Semantic Toolkit . . . . .     | 12        |
| 6.2       | Automated Systems Management . . . . .                         | 12        |
| 6.3       | Meaningful Programming Language . . . . .                      | 13        |
| 6.4       | Design Repository . . . . .                                    | 13        |
| <b>7</b>  | <b>Deterministic Intelligence</b>                              | <b>14</b> |
| <b>8</b>  | <b>Additional Thoughts</b>                                     | <b>15</b> |
| 8.1       | Automated Orchestration . . . . .                              | 15        |
| 8.2       | Securing Interactions . . . . .                                | 16        |
| 8.3       | Automated Documentation . . . . .                              | 16        |
| 8.4       | Research . . . . .                                             | 16        |
| <b>9</b>  | <b>Conclusion</b>                                              | <b>17</b> |
| <b>10</b> | <b>Extended Applications</b>                                   | <b>18</b> |
| 10.1      | Universal Design Language and Digital Twins . . . . .          | 18        |

# 1 Introduction

Software systems are an essential part of modern society and when they fail to function as intended, the implications can be enormous. It is therefore vital that systems are built to minimize the possibility of failure and that, when failures do occur, effective diagnostic processes expedite the systems recovery and prevent the recurrence of failures.

To address this need, this document presents a framework called the Design Learning Platform (DLP) that automates the development and management of intelligent software systems.

## 2 The Design

A design is a closed semantic world. It contains participants whose state is transformed through the design's unambiguous behaviors. When the environment interacts with the design, it introduces a change to the state of the world. This state transition may enable new behaviors, which the design deterministically resolves in response to the updated state, producing further state changes. When resolving a state, the design may either exhibit behavior directly or deterministically select which behaviors are enabled as a reaction to the state. The design continues reacting to each resulting state transition until no further behavior is enabled and the world reaches a stable semantic state. This chain of resolutions constitutes the structure of the design itself and provides the environment a fully specified path through it.



Figure 1: The Design

### 2.1 Meaning of the World

As part of the design, each behavior has an unambiguous meaning, it unambiguously transforms the participants and its attributes through the transformation. When a behaviors meaning is established as a composition of behaviors, it is called a composite behavior. The composite behaviors meaning is established by the semantics of the behaviors that compose it.

In Figure 2, the meaning of the behavior `placeBookOnShelf` is composed by the specified behaviors and the established control flow. This is no different than how words work as part of human communication, the word `lion` is a compression of the meaning of a lion, this includes the appearance, sound etc. A closed semantic world unambiguously establishes the meaning within its world. Through the composition of behaviors, each level of meaning establishes a coherent world.

#### 2.1.1 Shared Meaning

To motivate the need for shared meaning to have successful interactions, consider the following example about a library. A visitor enters the library and finds the book that they want to checkout. They walk up to the librarian with the intention to checkout the book. The librarian greets them with the intention



Figure 2: Composite Behaviors

of helping them checkout their book. The user hands them the book but the interaction halts because the librarian was expecting a library card first.

In this example, the interaction failed because of a lack of shared meaning about what it means to checkout a book. Since it is two humans interacting, the librarian can clarify to the user that the correct way to checkout a book is to first provide the library card. The visitor updates their definition of checkout book and the interaction proceeds successfully with shared meaning. This example demonstrates how shared meaning is essential for the success of an interaction.



Figure 3: Semantic Compatibility through Shared Meaning

This same principle applies when two designs are interacting, the success of their interaction is dependent on their ability to understand each other. When two designs interact with each other and have shared meaning, the interaction will succeed. Through shared meaning, they form a larger semantic world built through semantically compatible interactions. This process will establish the design of a distributed system assembled through semantically compatible interactions forming a larger closed semantic world with unambiguous meaning.

Figure 3 demonstrates this using a distributed image compression system. For example, when the database is designed, the meaning of the database user is established. When the image compression service interacts with the database, it assumes the role of the database user, thus establishing shared meaning with the database.

The ability to construct larger closed semantic worlds through semantically compatible interactions between designs is meaningful because all the techniques developed in this document naturally extend to the larger world.

## 2.2 Narrative of the World

Another way of describing a closed semantic world is through the narratives that are established by the design. The narratives are constructed using the behaviors and how it unambiguously modifies the state of the world's participants. An example of a narrative could be: "The librarian accepted the book from the user and added the book to the basket". This way of speaking about the design captures the meaning of the world in a form that is very intuitive. More importantly, the meaning of the narrative is derived from the constructed world. Consequently, the inverse is also true, the world can be automatically constructed from a meaningful narrative.

## 2.3 Semantic Invalidity

Semantic invalidity can be generally understood as incorrect or missing meaning. In this sense, the world is in a semantically invalid state when one or more of the following happens:

- The world's meaning does not satisfy the requirements of the design.
- The intentions of the design cannot be realized due to the state of the world.
- The intentions of the design cannot be realized because reality refuted the design's assumptions.

This means that the correctness of the design can be established by identifying if it eliminates the known semantically invalid conditions. The first step is to ensure that the meaning of the world is correct with respect to its requirements. This is a foundational step because it establishes the correct meaning that the rest of the process builds on.

After ensuring that the requirements are satisfied, when the environment provides an input into this closed semantic world, in order to continue successfully realizing its intentions, the design must prevent semantically invalid states from persisting in its world.

To prevent semantically invalid states from persisting in its reality, semantic invariants act as markers that identify the semantically invalid state and predict the failure of downstream behaviors. This establishes an invariant path from the semantically invalid state to the intention that can't be realized and in order to restore semantic validity, the design must provide a semantically valid path to resolve the state.



Figure 4: Semantically Invalid Narrative and Semantically Valid Narrative

In order for the closed semantic world to participate in reality, it must be implemented on a substrate. When realized on a substrate, reality can refute the assumptions of the closed semantic world and the design fails because its intentions can't be realized. As a result, the design must learn new semantics and continue to prevent semantically invalid states from persisting in its world to continue realizing its intentions.

In the next section, I will describe how by establishing the design in a structured and unambiguous form and engine can reason about the world. In the process, I will define the world failure and explain how the engine can diagnose failures through semantic reasoning.

### 3 Semantic Reasoning Engine and Diagnosing Failures

The design is a structure that can be reasoned about through the meaning that it establishes. In this context, reasoning means to ask meaningful questions about it to extract a meaningful answer. As such, if the design is established in a structured and unambiguous form, an engine can automatically reason about the world.

#### 3.1 Meaning of Failures

Reasoning about the world is a familiar process to developers because it has always been a part of software development. This is because software systems have always been the design of a closed semantic world. The process of reasoning about the world is asking meaningful questions about it to get meaningful answers.

When a meaningful question cannot be answered, it exposes missing meaning and in a closed semantic world, this is the meaning of a failure. As such, in this document, the world failure means an inability by the engine to answer a meaningful question about the world.

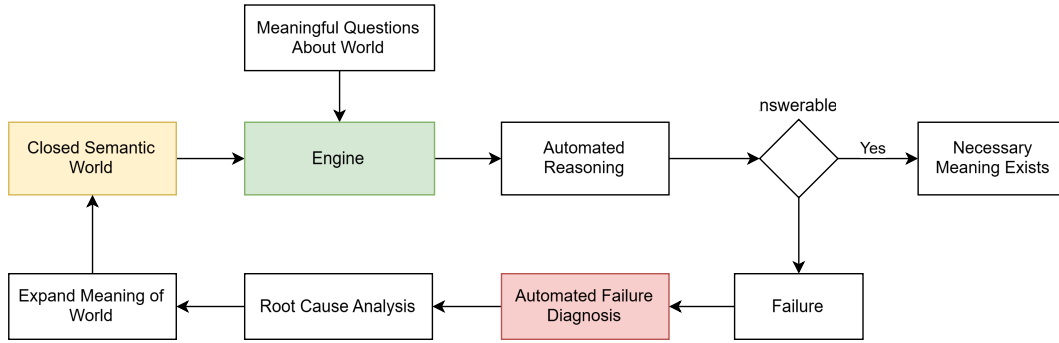


Figure 5: Automating Failure Diagnosis by Reasoning about World to Answer Meaningful Questions

The word failure has historical baggage because it has often been used to reference mechanical failure experienced by the implementation of a design on a substrate (which is one possible result of semantic failure). So I want to make that distinction and clarify that the use of the word failure in this document is not to be confused with mechanical failure. It is worth highlighting this explicitly because understanding what failures are in a semantic world is the heart of this solution and the way it frames software systems.

In earlier sections, I established the conditions under which semantic invalidity exists. The persistence of the semantic invalidity can be captured through either an inability to answer a meaningful question about the world or an inability to identify the meaningful question entirely. In the following sections, I will describe how semantic invalidity is eliminated through reasoning by expanding the semantics to identify or answer meaningful questions to eliminate failures.

#### 3.2 Design Abstraction Language

To make the reasoning possible, the design must be defined in a Design Abstraction Language (DAL). The DAL is a declarative language that is used fully establish the closed semantic world. The scope of the DAL includes all the meaning needed to fully define the world (behaviors, transitions, participants, invariants, etc).

Later in the document, I will describe a semantic toolkit that can be used establish the meaning of a world. This meaning is automatically translated into a closed semantic world defined in the Design Abstraction Language (DAL) and the engine reasons about the world using the structure until the meaning is correct. The established meaning can then be added to the semantic toolkit and the process repeats.

#### 3.3 Types of Reasoning

An engine can use the DAL definition to automatically reason about the world. When the engine is unable to answer a meaningful question about the world, the failure is automatically diagnosed because it identifies where the semantics have to expand through root cause analysis.

Figure 5 demonstrates the general structure of how the engine can automatically reason and expand the meaning of the world by learning new semantics. In the following sections, I will demonstrate how this reasoning is applied to learn from the requirements, the design own meaning and reality.

### 3.3.1 Reasoning about Requirements

By answering meaningful questions about the requirements, the engine can reason about the world and identify where the meaning of the world has to expand through root cause analysis. This feedback loop converges when the meaning of the world fully satisfies the requirements.

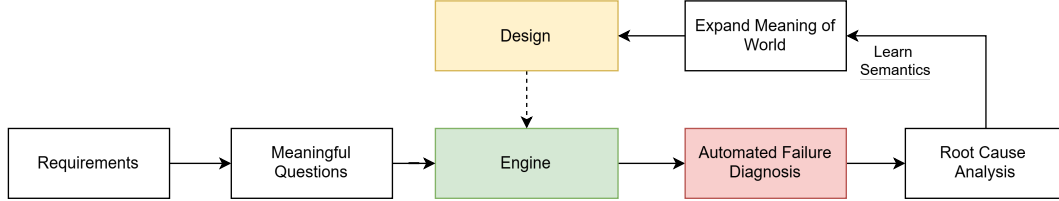


Figure 6: Reasoning to Verify Requirement Satisfaction

This is possible because the requirements and the design share meaning. If the meaning of design satisfies all the requirements, then the necessary conditions to verify the internal consistency of the design will exist. It also becomes clear that if the requirements themselves are ambiguous, then the design will inherit this ambiguity.

However, this also implies that if the requirements are specified in a structured and unambiguous way, the design can be automatically generated from the requirements. When the completeness of the requirements converges on its limit, it becomes the meaning of the design itself. This will eliminate the need to reason about the requirements to establish the necessary meaning in the world because the meaning itself will be specified and used to construct the world.

In this process, building the design itself becomes an automated reasoning process, where the meaning is used to automatically construct the world and the engine identifies the missing meaning that prevents internal consistency. Through this process, the development of software systems becomes automated because the meaning is automatically realized as an implementation.

### 3.3.2 Reasoning about Internal Consistency

As the previous section described, the closed semantic world is automatically constructed using the specified meaning. As a result, using this meaning, the engine can then reason about the internal consistency of the design.

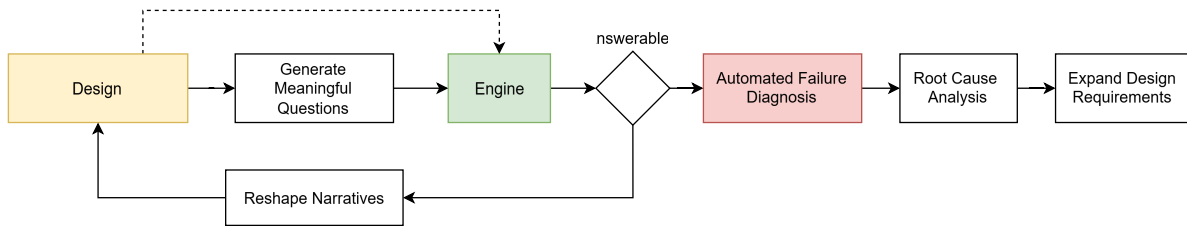


Figure 7: Automated Reasoning to Reshape Narrative

Through the meaning of the world, it is possible to identify semantically invalid narratives. For example, if a narrative gets a book from the basket but the world allows narrative to be exhibited when the basket is empty, then this will result in a failure as predicted by the invariant. This can be resolved by selecting a semantically valid narrative to prevent the invalid narrative from being selected. This process is demonstrated in a general way in Figure 4.

However, if the necessary meaning doesn't exist in the world to be able to reason about the designs internal consistency, then the failure is automatically diagnosed and the subsequent root cause analysis expands the

meaning of the world. For example, if the invariant specifies that the basket size has to be at least one before you try to get a book from it, and the design doesn't know what basket size means, it will be unable to reason about it. In this way, the meaning of the world will expand until the engine can eliminate all semantically invalid narratives.

While this is a simple example, it highlights a more important point that inconsistencies in the constructed narratives will be identified by the reasoning. If there is no narrative that adds a book to the basket, then is the get book from basket narrative actually needed or is the behavior to add a book to the basket missing? The engine can reason about this because it recognizes that the world state never modifies the basket size, so you can never get a book from the basket.

In this sense, this process will actually expand the requirements of the design until it is complete and the internal consistency can be verified.

### 3.3.3 Reasoning about Reality

Since the design establishes its assumptions unambiguously, the engine can reason about reality to determine when the reality refutes the design's assumptions. Since it will be unable to explain why reality refuted the design's assumptions, this will result in automated failure diagnosis and the design's semantics will expand to explain why. Mechanical failures are just one way in which reality can refute the design's assumptions, any assumptions made by the design like timing requirements can also be refuted.



Figure 8: Ideal learning loop by reasoning about execution using design meaning

In this process, the design learns how to function effectively in its operational environment and becomes progressively more intelligent. This happens through the acquired environmental invariants which expand the design's awareness of its environment.

However, in order to practically enable this process of learning from reality, the execution of the design must be losslessly preserved so that the engine can reason about it using the model's assumptions. In the next section, I will discuss the practical issues that must be considered.

## 3.4 Practical Considerations for Reasoning about Reality

In order for the engine to reason about the execution to learn new semantics, the following conditions must be met:

- The implementation and its execution must represent the design semantics faithfully.
- The execution must be losslessly preserved for the reasoning to be deterministic.

If the semantic gap between the implementation and the design is eliminated and the execution is losslessly preserved, it would enable a deterministic learning loop. More importantly, as a result of this, any meaningful question about the world can be automatically answered or an automatic process would collect the necessary diagnostic information needed to learn how to answer the question.

In the following sections, I will explain how a Computable Semantic Model (CSM) can be used to eliminate the semantic gap between the design and the implementation. Then I will explain how domain specific compression can be used to losslessly preserve all the narratives experienced in the world so the engine can deterministically reason about it and finally I will explain how all of this works to fully automate systems management.

## 4 Computable Semantic Model

In order for the engine to reason about the execution of the design, the semantic gap between the design and the implementation must be eliminated. If this is not done, then the engine cannot deterministically reason about the world because there will be no way to know if the semantics need to expand or if the implementation doesn't realize the meaning of the design.

If the implementation does realize the meaning of the design faithfully, then all the automation that is enabled by the engine reasoning about the design can be leveraged. To make this possible, the design must be established as a Computable Semantic Model (CSM) that is built from Computable Semantic Primitives (CSP) and I will describe how in the following sections.

### 4.1 Implementing a Computable Semantic Model

A CSM is built by establishing the implementation of the designs meaning enabling the implementation of the design to be directly synthesized into an executable form. The design establishes the first behavior it exhibits and then the design reacts to each impulse from the environment until it reaches a semantically stable state.



Figure 9: Computable Semantic Model

However, there is still the possibility that the provided implementation doesn't actually realize the meaning of the design, in which case, the engine cannot reason about the world in a reliable way. Moreover, the engine can only reason about the world based on the meaning that was specified. So unless the design makes all the relevant semantics available, the engine will be unable to determine the worlds correctness. So the engine will know that there is a failure and it won't be able to explain why until it learns the missing semantics.

In the next section, I will describe how designs built from Computable Semantic Primitives addresses both of these problems.

### 4.2 Computable Semantic Primitives

Computable Semantic Primitives (CSP's) are primitive computable semantic units that can be used to construct the meaning of the semantic world. Since the world is constructed from meaningful atoms that have unambiguous invariants, it can be reasoned about completely to eliminate all semantically invalid narratives. Since the primitive units can be synthesized into an implementation by definition, this transforms the DAL into a declarative language that establishes the meaning of a closed semantic world and synthesizes the meaning of that world as an implementation, effectively making the design itself executable.

As previous sections mentioned, the engines ability to reason about the execution of the world is dependent on there being no semantic gap between the design and the implementation. Using CSP's solves this problem elegantly in multiple ways:

| Meaning of Closed Semantic World |     |          |          |     |          |     |          |     |          |
|----------------------------------|-----|----------|----------|-----|----------|-----|----------|-----|----------|
| Behavior                         |     |          |          |     | Behavior |     |          |     |          |
| Behavior                         |     | Behavior |          |     | Behavior |     | Behavior |     | Behavior |
| CSP                              | CSP | Behavior | Behavior |     | CSP      | CSP | CSP      | CSP | CSP      |
|                                  |     | CSP      | CSP      | CSP |          |     |          |     |          |

Figure 10: Computable Semantic Model constructed from Computable Semantic Primitives

1. It allows the engine to fully reason about the constructed world to eliminate any semantically invalid narratives.
2. It allows the engine to reason about the execution to automatically diagnose any failures and motivate the necessary root cause analysis to learn new semantics.

Now that the constructed model and the execution have the same meaning, the engine can automatically identify where the semantics need to expand because reality will refute the designs assumptions through failures.

However, this automation will still not be possible unless the engine actually has a complete account of the execution to diagnose the failure. In the next section, I will talk about the information that needs to be preserved to make this possible.

### 4.3 Automated Instrumentation

Since the design is an unambiguous structure, it isn't necessary to preserve all the execution information to be able to replay it. The design itself can identify the information that can't be deterministically computed by the design. This means that the synthesized implementation can be automatically synthesized to only instrument the non-deterministic information.



Figure 11: Automated Instrumentation and Synthesis

As Figure 11 illustrates, the engine can use the unambiguous specification of the CSM to identify the necessary instrumentation. The information produced by the execution is then fed back into the engine where the failure is automatically diagnosed. In this way, the ways in which reality refutes the assumptions made by the design can be unambiguously identified and the learning process can be triggered.

However, to truly make this an automated process, all the diagnostic data needs to be preserved at scale. Any missing data would introduce speculation, so there is a need for a solution address the practical problem while minimizing cost. In the next section, I will describe how this can be addressed by using the unambiguous domain structure of the data to apply domain specific compression.

## 5 Domain Specific Compression

In order to preserve all the diagnostic data necessary for the engine to deterministically reason about the world, the unambiguous domain structure of the CSM can be leveraged to apply Domain Specific Compression (DSC). This technique leverages the known domain structure to optimally compress it.

### 5.1 Compressed Log Processor

Practically, this can be achieved using a tool named Compressed Log Processor (CLP). It is an open source tool that can apply DSC and search the compressed data without decompression. More importantly, it has proven that DSC is practical at a petabyte scale. Currently, CLP compresses structured and unstructured logs by exploiting their known domain structure. However, in its current form, the domain structure of the variables in those logs is not known, so CLP is not maximizing compression it can achieve.



Figure 12: CLP Logo

Using the CSM, CLP can be transformed into a DSC engine. CLP's gains come from a lack of ambiguity and the CSM works to eliminate ambiguity entirely in its domain, thus CLP's compression will be maximized. More importantly, since the data exists as part of a structure with unambiguous meaning, the compression will improve further by exploiting the repetitive and predictable nature of programs.

When the engine replays the execution using the collected diagnostic data to automatically diagnose the failures, the narratives that are played out in the semantic world can be preserved by CLP at minimal cost. This means that the engine will never have to re-compute the same transformation twice. Instead, it can leverage CLP's search without decompression feature to identify if the narrative's transformation has already been computed.



Figure 13: Overview of CLP's current solution.

This creates a very powerful ability where CLP contains the entire experiential history of the closed semantic world. When combined with the CSM, this is what allows the engine to reason about the world deterministically. CLP's search performance will also become more optimized since the both the structure and the meaning of the data are entirely unambiguous. The queries themselves are inherently semantic because the data exists as part of a closed semantic world and the queries can be meaningfully structured.

Ultimately, this will result in a fully optimized platform because the meaning of any data in the platform is inherently known. Moreover, when CLP is combined with the CSM and the engine, this results in a design learning platform that fully automates software systems management.

# Design Learning Platform



Figure 14: The Design Learning Platform

## 6 Design Learning Platform

This section will present the Design Learning Platform (DLP) and describe how it automates the development and management of software systems. Many of the ideas presented here have been established in earlier in this document but this section brings them all together under one tool presented in Figure 14.

### 6.1 Automated Development using the Semantic Toolkit

In this framework, the programming language is meaning and this can be referred to as a semantic toolkit. The semantic toolkit is used to establish the meaning of the world and this meaning is then automatically constructed as a closed semantic world. When the meaning of the world is incomplete, the constructed world will be internally inconsistent and there will be many worlds that can realize the established meaning. As the meaning reaches completeness, the constructed world becomes internally consistent and the number of worlds that can realize this meaning collapse into one.

The semantic toolkit consists of the computable semantic primitives that establish the atomic meaning used to construct the worlds. Initially the meaning of the world is expressed through narratives constructed from the semantic primitives and a closed semantic world can be constructed that realizes this meaning. Since the CSP's that are used to construct this world can be synthesized into an implementation, the semantic gap between the constructed closed semantic world and the synthesized implementation is eliminated, making the meaning itself executable.

As more meaning is constructed, the semantic toolkit expands and the kinds of narratives that can be constructed also expand. This means that even as the meaning of the world expands it can still be synthesized into an implementation because the meaning can also be reduced down to its atomic building blocks. However, the world constructed from the specified meaning can be internally inconsistent or incomplete, resulting in failures.

To address this, the meaning of the world can be reasoned about by an engine to automatically diagnose the failures, this is then used to refine the meaning until it is complete. As earlier sections established, in this framework, a failure is an inability to answer a meaningful question about the world, so by reasoning about the world, the engine exposes missing or incorrect meaning. So building a software system becomes a process of establishing its meaning using a semantic toolkit, automatically constructing a world that realizes the meaning and using the engine to reason about the world so it can be automatically managed. In this process, this automates the development and management of software systems.

In the next section, I will speak more specifically about how the engine automates the management of the constructed world and how the meaning of the world is refined by reality when it refutes the designs assumptions.

### 6.2 Automated Systems Management

As the previous section explained, the engine then reasons about the world constructed from meaning constructed using the semantic toolkit to identify internal inconsistencies and refines the requirements until the world is internally consistent. Since the world is built from computable semantic primitives, it can be synthesized into an implementation whose meaning has no semantic gap with the provided meaning.

When reality refutes the assumptions made by the design, new semantics are learnt through root cause analysis and used to refine the meaning of the world. The meaning is then once again synthesized into a world that is internally consistent and has the correct meaning. To learn semantics through root cause analysis when reality refutes the designs assumptions, the execution history of the design is losslessly preserved at scale by a tool named CLP using domain specific compression.

Ultimately, this automates the management of software systems because the automated reasoning constructs a world that has already been managed by construction. The necessary questions about the world have been asked and answered by the engine until the world is internally consistent and its meaning is correct. When the system still fails, an automated process provides the necessary information to support root cause analysis to learn new semantics.

Practically, building the design itself becomes a process of world building from meaning where the feedback from the engines automated reasoning refines the requirements until the world is internally consistent and

has the correct meaning. This will result in a series of questions that must be answerable by the engine to establish that the meaning of the world is correct and is internally consistent. If the meaning of the world is correct with respect to its requirements and it is internally consistent, then there are no more known questions that have to be asked with respect to managing the system. When the system fails, the framework learns a new question that the engine must be able to answer and the world gains the meaning needed to be able to answer it.

If the requirements specify a design that doesn't actually establish the world that serves the purpose of the design, then there is no way for the engine to know if its meaning is correct. However, it is evident from this structure that the automated reasoning is only limited by the meaning that has been established. So you can keep moving up the hierarchy and establish whether the requirements describe a world that serves the purpose of the design. Then the entire world will be reasoned about from the purpose down to the implementation while ensuring its correctness.

### 6.3 Meaningful Programming Language

This automates the processes involved in software development by eliminating any ambiguity in its intended meaning and then using that meaning to construct the world, reason about it and synthesize it into an implementation that has no semantic gap with the intended meaning. This results in a new programming language that is inherently meaningful and can be described as a semantic toolkit. I have yet to decide on a name but I explored calling it Semkit or Artha (which means meaning).



Figure 15: Automation achieved by Inverting Software Engineering

This programming language inverts traditional programming, rather than using code to realize the meaning, the meaning is used to automatically generate the implementation. The ability to establish a meaning first programming language changes the nature of programming and allows the developer to create intelligent systems that are inherently aware of their own meaning. As the semantic toolkit expands, it creates new possibilities where systems coordinate by understanding each other and as a result the intelligence of the coordination can be maximized.

### 6.4 Design Repository

Traditionally a code repository contains the evolution of the implementation of the program and the meaning of the evolution is limited to the commit messages or the pull requests that describe the commit.

However, with this solution, since every environment that motivated new semantics is preserved, it results in a design repository through which the evolution of the software system's meaning can be deterministically replayed. The repository tracks not just the changes to the implementation but also how the meaning of the system, its synthesized implementation and how it was executed and refined through learnt semantics over time. This is a natural conclusion of eliminating any ambiguity in the construction and of a software system and in the execution of that constructed system, it eliminates ambiguity in its evolution.

## 7 Deterministic Intelligence

In this section, I will explore how this framework establishes deterministic intelligence. This is a consequence of inverting software engineering by defining the meaning of the world through the programming language and realizing the implementation of that meaning. It is ultimately just a program but since it is reasoning about meaning during execution, it becomes intelligent. This form of intelligence remains fully debuggable because its reasoning is fully traceable. The effectiveness of the intelligence is determined by the reasoning established in the narratives of the world.

Intelligence can be perceived as ability to reason about the meaning of a world. Through the processes established in this document, a semantic toolkit is used by a programming language to specify the meaning of the software system, transforming its execution into a process of reasoning about the world. As a result, software systems are inherently intelligent because they are the fixed set of narratives being realized by an intelligent mind that is reasoning about the world.

This means that when software systems coordinate through shared meaning, it is two intelligent entities coordinating to achieve their intentions. This is a form of deterministic intelligence that can be fully debugged. Initially, this is a rather difficult to internalize because ultimately it is still a software system, so how can it suddenly be intelligent? The reason is because the mechanics are now explicitly represented as meaning that is reasoned about, this results in the software system exhibiting deterministic intelligence. It is reasoning about the world and expressing meaning that is being realized by the mechanics.

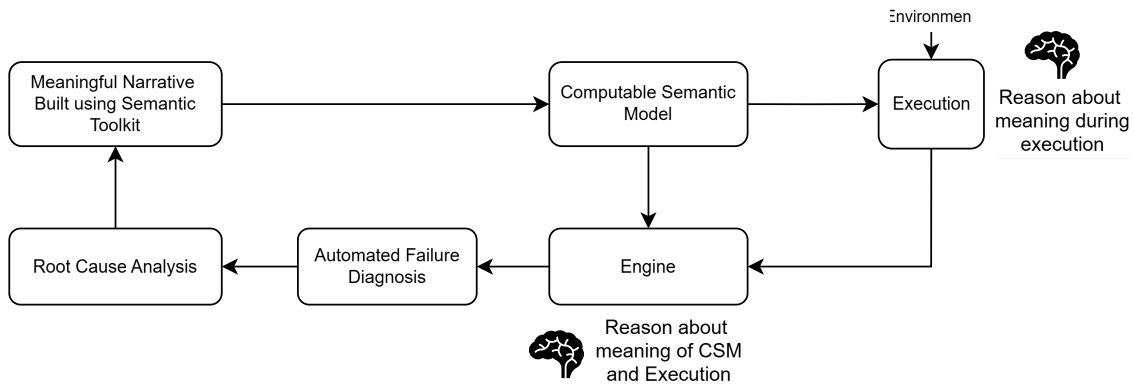


Figure 16: Intelligence as Reasoning about Meaning

The engine is also intelligent because it is reasoning about the world to establish if it has the correct meaning either from the CSM or the execution. It automatically diagnoses failures by reasoning about the world and this is another form of deterministic intelligence.

The following list captures the current forms of deterministic intelligence in this framework:

- Engine reasons about meaning established by structure the closed semantic world.
- Engine reasons about meaning from execution of the closed semantic world.
- Execution reasons about meaning of the world through the narratives of the closed semantic world.

With software systems, the programming language is creating a fixed set of narratives that can reason about the world and the synthesized implementation of the program realizes the intentions of those narratives. Its not that there is a literal intelligent mind operating the software system, its that the reasoning established in the narratives are a fixed set of behaviors that an intelligent mind would exhibit by reasoning about the world. The deterministic intelligence and its effectiveness is a result of the reasoning that exists in the fixed set of narratives that are established in the closed semantic world. A better way to say this is that the meaningful programming language allows intelligence to be constructed.

Ultimately, the transition from writing mechanical movements to writing the meaningful narratives that reasons about the world will transform the way that software systems are developed. It reframes software engineering as the process of constructing and expressing intelligence and as a result, this enables the systems of the future.

## 8 Additional Thoughts

This section contains additional thoughts relevant to this framework but couldn't be added to the rest of the sections.

### 8.1 Automated Orchestration

Automated orchestration falls directly from the ability to build computable semantic models which have shared meaning. When a distributed system is defined, it also defines an orchestrator that has shared meaning with the design. It is through this shared meaning that orchestration is made automated and surgical because the state of the system being managed is unambiguous by design.

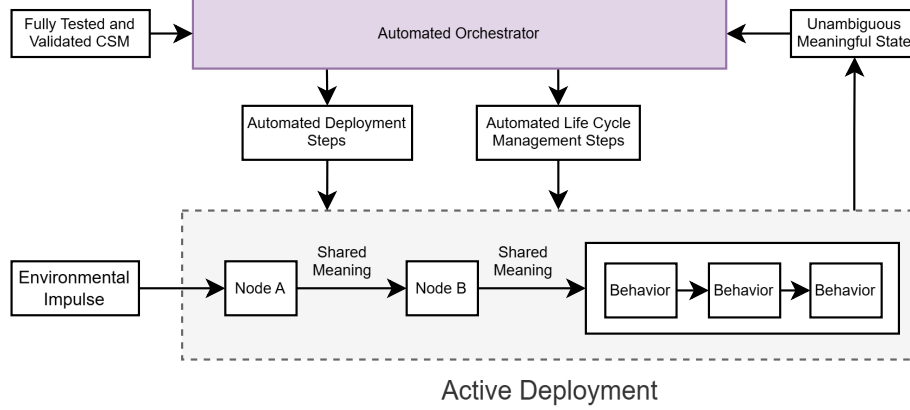


Figure 17: Automated Orchestration of Distributed System

Figure 17 shows a block diagram how of orchestrator can react to an unambiguous meaningful state to take the precise response because it knows the meaning of the CSM that it is managing by design.

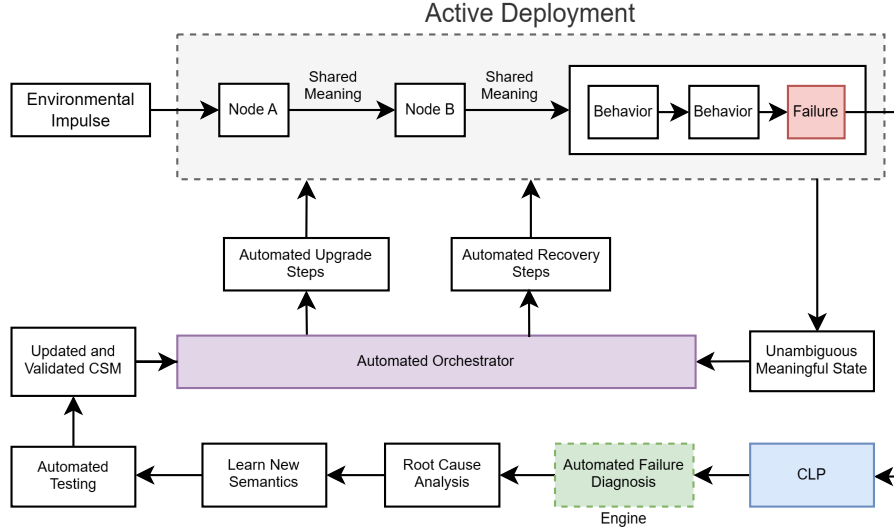


Figure 18: Automated Orchestration of Recovery from Mechanical Failures

Figure 18 shows a block diagram of how an orchestrator can react to a failed state and take the necessary steps to recover the system. This is once again made possible because the failed state has unambiguous meaning to the orchestrator by design. At the same time, the learning loop described in a previous section works to ensure that the failure is automatically diagnosed and a deterministic learning process prevents the recurrence of the same failure.

Ultimately, this isn't anything special, it is just an extension of the closed semantic world to include the meaning which establishes the conditions needed for the design to realize its intentions. Through semantically

compatible interactions, the same principles that establish the correctness of a design apply here. This also means that the testing of the orchestrators response to meaningful states in the design can be automated.

## 8.2 Securing Interactions

As a result of this framework, interactions between and within designs now has unambiguous meaning. There are meaningful implications for securing systems because the intention of the interaction is unambiguous. While before, the mechanical sequence had to be correct, now the design can ensure that it is the right person that is communicating with it. If the design is looked at as the brain of the software system, then this ensures that the correct brain is speaking, making it more difficult to compromise a system.

There are many approaches to implement this but one possibility is that DAL can be extended to generate a dynamic checksum along the narrative path to identify the unique brain that generated the narrative. Unlike traditional techniques which have to speculate about whether the intentions of a particular sequence are suspicious through practices like anomaly detection, this solution would automatically recognize suspicious intentions from construction. The security is built into the narratives that are permitted in the closed semantic world.

There are also automated ways to test and verify how secure a system is by following the same principle of eliminating ambiguity using the CSM to answer meaningful questions about the system. A future where software systems understand each others intentions and meaning will be a more secure future.

## 8.3 Automated Documentation

Traditionally, documentation was the developers best attempt to capture the design of the system. This was then used by other developers to establish the necessary semantics to interact with the design correctly. It is clear that this relies on the skill of the developer heavily and can introduce bugs through human error.

Through the processes established in this document, the documentation has become automated because the design is able to represent itself. More importantly, the designs themselves can verify that there is semantic compatibility between each other by ensuring that the necessary meaning has been established on both sides. In this sense, the documentation is the design and since it is representing itself, the documentation itself is used to verify the correctness of the semantics.

By enabling the CSM to represent itself unambiguously as the documentation, software systems are made more reliable because it minimizes the possibility of errors.

## 8.4 Research

In this section, I will briefly discuss this novelty of this solution from a research perspective. It is difficult to call the CSM a breakthrough because it is simply formalizing the existing software engineering practices into a computable model so that it can be automatically reasoned about by an engine. However, it is novel in the sense that this capability did not exist before and it is a necessary capability to automate the development and management of software systems. In building the CSM, it establishes a meaningful programming language and that does have significant implications because it enables the development of intelligent systems that understand each other, this opens the door for a whole new class of systems.

Domain specific compression is also a necessary capability to make the automation possible because it overcomes a hurdle that prevented automated systems management. Until CLP existed and proved that domain specific compression was possible at a petabyte scale, there was no solution that could address this need or even establish that the problem was solvable. In this sense, the novelty of CLP is that it proved that the preservation of the information necessary to automate systems management is possible at scale. So from a research perspective, the maturity of domain specific compression was a real breakthrough that made automated systems management possible.

However, neither the CSM, the engine or CLP solve this problem on their own. They all exist as part of a coherent solution that works to automate the development and management of software systems. By transforming the development of software systems into a process of meaningful world building, this lays the foundation for many new areas of research.

## 9 Conclusion

Traditional software engineering uses the mechanical abstractions of a programming language to realize the meaning of a software system. The techniques presented in this document take a different approach, they specify the meaning of the software system and use it to automatically construct a computable semantic model from the meaning. Since the computable semantic model is built from computable semantic primitives, it can be directly synthesized into an implementation that has no semantic gap with the specified meaning of the system.

Prior to synthesis, an engine is used to reason about the CSM's meaning to identify if it is missing meaning or is internally inconsistent. It automatically diagnosis failures by identifying meaningful questions that can't be answered by reasoning about the CSM and motivates root cause analysis to learn new semantics. In addition to internal inconsistencies, the designs semantics also expand by enabling the engine to automatically diagnose failures when reality rejects the designs assumptions. This is done by instrumenting the synthesized program to gather necessary diagnostic data, preserving it at scale by using CLP to apply domain specific compression and replaying the execution to allow the engine to reason about it. In the process the design learns new semantics and becomes progressively more intelligent as its meaning expands to continue realizing its intentions in its operational environment.

As is evident, in this framework, meaning is the primary attribute that drives the entire process. Any process that reasons about meaning and makes decisions is considered to be intelligent and in this case, the narratives established with the semantic toolkit contain the intentions of intelligence that reasons about the world during execution and makes decisions. This makes the software system inherently possess intelligence because its meaning is defined through a fixed set of narratives that reason about the world to achieve their intentions. The effectiveness of the intelligence is determined by the established reasoning in the narratives of the world. Further more, the engine itself is intelligent because it reasons about the worlds meaning to perform its function.

In this process, the development of software systems is automated by using a semantic toolkit to automatically build the closed semantic world and its implementation from the specified meaning. The management of software systems is automated by reasoning about the world to automatically diagnose failures and ensure its correctness through construction. Every meaningful question about the world has been asked and answered, enabling the design to learn new meaningful questions through root cause analysis on failures. Through this process of answering every meaningful question and deterministically learning new questions and answers, the software system is automatically managed.

Finally, as a result of the processes described in this document, the software systems that run modern society are less likely to fail and if they do fail, an automated process ensures their recovery and automated failure diagnosis supports deterministic learning to prevent future failures. This framework transforms the construction of software systems into a process of meaningful world building while using an engine to reason about its correctness. The future of software systems will be less about writing bug free code and more about building intelligent and secure systems that understand each other.

## 10 Extended Applications

In this section, I present some applications that can immediately benefit from this framework outside of software systems.

### 10.1 Universal Design Language and Digital Twins

Designs are the universal structures used across many disciplines. The ability to unambiguously establish a closed semantic world and reason about it has applications well beyond software systems. If implemented with the right strategy, this framework has the ability to become a universal design language used in many disciplines.

Since the information contained in the DAL has unambiguous meaning, it can be used to establish a powerful visualization framework. This would allow engineers to visualize the closed semantic world in a meaningful way and gain insights into their design by reasoning about its meaning. More importantly, since this framework extends the design into a computable semantic model (CSM), it extends the CSM to become a digital twin of what it is modelling.

In software systems, the design establishes a closed semantic world that mirrors what participates in reality because the design is what defines it (this statement is even more valid with CSM's constructed using CSP's). This is a unique property of software systems since they operate entirely on a stack that is symbolically defined by humans. However, in other disciplines, reality is modelled by engineers to be able to represent it accurately and predict it. In these applications, the CSM becomes the framework through which that modelling happens.

In the digital twin models, information collected by sensors is used to inform the model and refine it so that it is a more accurate representation of reality and to improve its ability to make more accurate predictions. In these cases, the domain structure of the collected data is unambiguously established by the CSM, as such, the data can be domain compressed using CLP's engine. This extends the use of CSM and CLP beyond software systems.

The applications of CSM are only limited by what can be modelled, anything that can be modelled as a closed semantic world can be reasoned about using the engine. A personal favorite of mine is to model concepts using the CSM so that the model can be used as a teaching tool. This application exploits the fact that when humans teach, they are ultimately building a closed semantic world to convey the concept to another human mind. Using the CSM, the semantic world necessary to represent the idea can be transformed into a computable model in which every step in the process is informed and validated by the model. There are many interesting ways in which this can be used to turn teaching and learning into an unambiguous process that converges on complete understanding.